Structural design patterns focus on how classes and objects are composed to form larger structures, and all three tasks here show that principle in slightly different ways (Francesco, 2024, ch. 2).

Adapter Pattern

A good real world case is plugging a legacy SOAP based payment system into a modern e-commerce platform that only speaks REST and JSON. Rewriting the legacy system is expensive and risky, so instead an adapter class sits between the two, converting JSON requests into the SOAP format the old system expects and translating the response back again. The e-commerce platform never knows the legacy system exists underneath.

Python code

```
class LegacySoapPayment:
    def make_soap_payment(self, xml_data):
        return "<response>SUCCESS</response>"

class PaymentAdapter:
    def __init__(self, legacy_system):
        self.legacy_system = legacy_system

    def pay(self, json_data):
        xml_data = f"<payment>{json_data['amount']}</payment>"
        response = self.legacy_system.make_soap_payment(xml_data)
        return {"status": "SUCCESS"} if "SUCCESS" in response else {"status": "FAILED"}
```

Bridge Pattern

Think about a universal remote app that has to control a TV and a Radio, each with Basic and Advanced remote variants. Without Bridge you would end up with a class for every combination. Bridge splits the remote (abstraction) from the device (implementation) so either side can change independently.

Python code

```
class Device:
    def turn_on(self): pass

class TV(Device):
    def turn_on(self): print("TV is on")

class RemoteControl:
    def __init__(self, device):
        self.device = device

    def power_on(self):
        self.device.turn_on()
```

Composite Pattern

A file system is the textbook example. Files and folders both need a size and a display method, and a folder can hold either. Composite lets you call the same method on a single file or an entire folder tree without checking types first.

Python code

```
class FileSystemComponent:
    def show(self): pass

class File(FileSystemComponent):
    def __init__(self, name):
        self.name = name
    def show(self):
        print(self.name)

class Folder(FileSystemComponent):
    def __init__(self, name):
        self.name = name
        self.children = []
    def add(self, component):
        self.children.append(component)
    def show(self):
        print(self.name)
        for child in self.children:
            child.show()
```

Reference

Francesco, H. D. (2024) *JavaScript Design Patterns: Deliver fast and efficient production-grade JavaScript applications at scale*. 1st edn. Birmingham: Packt Publishing.